

TMHCC DXLibrary ClaimForm — Component Documentation

TMHCC_DXLibrary_ClaimForm — Complete Component Documentation

- Table of Contents
- 1. Business Overview
- 2. High-Level Architecture
- 3. File Structure Reference
- 4. Component Props and Design-Time Configuration
- 5. Form Steps, Fields and Validation Rules
- 6. Validation System — Deep Dive
- 7. Pega and PCore Integration Points
- 8. Localization System
- 9. Attachment Upload System — End to End
- 10. Hero Image Loading via AssetLoader
- 11. Security Controls
- 12. Accessibility Implementation
- 13. Style Architecture
- 14. Developer Guide — How to Make Changes
- 15. Understanding the Pega DX Custom Component Pattern (Beginner Guide)
- 16. Storybook and Local Development
- 17. Appendix — Localizations Key Reference

TMHCC_DXLibrary_ClaimForm — Complete Component Documentation

Version 0.0.1 | **Pega Infinity** 25.1.2-397 | **Cosmos React** 8.21.0 | **Build** 2026-06-09
Organization TMHCC | **Library** DXLibrary | **Type** Template (PAGE / CASE)

Table of Contents

1. Business Overview
 2. High-Level Architecture
 3. File Structure Reference
 4. Component Props and Design-Time Configuration
 5. Form Steps, Fields and Validation Rules
 6. Validation System — Deep Dive
 7. Pega and PCore Integration Points
 8. Localization System
 9. Attachment Upload System — End to End
 10. Hero Image Loading via AssetLoader
 11. Security Controls
 12. Accessibility Implementation
 13. Style Architecture
 14. Developer Guide — How to Make Changes
 15. Understanding the Pega DX Custom Component Pattern (Beginner Guide)
 16. Storybook and Local Development
 17. Appendix — Localizations Key Reference
-

1. Business Overview

What This Component Does

`TMHCC_DXLibrary_ClaimForm` is a self-contained, multi-step web claim intake form embedded inside a Pega Infinity portal. It enables military kit insurance policyholders (and their family members or representatives) to report a loss event and submit a claim entirely online — without calling the claims team.

The form guides the claimant through four data-collection steps, then a review and declaration screen, and finally a confirmation screen. On submission, it creates a Pega case of the configured case type and associates all uploaded evidence files.

Business Value

Value Driver	Detail
Self-service intake	Claimants complete the form without agent intervention, reducing inbound calls
Structured data capture	All claim fields are mapped to specific Pega case properties, eliminating manual data entry by claims handlers
Document collection	Files uploaded by the claimant are immediately attached to the case in Pega's document management system
Fraud controls	Two mandatory legal declarations (fraud prevention notice and claim accuracy confirmation) must be checked before submission
Multi-country support	Country-aware postal code labels and validation (GB/US/CA/JP) handle international policyholders
Brand-consistent UI	Custom-styled form matches TMHCC brand guidelines while sitting inside any Pega portal page
Help contact panel	A fixed hero panel shows telephone, email and support hours, reducing "where do I get help?" contacts
Localization-ready	Every user-facing string is configurable via Pega's localization system without a code change

Target Users

- **Primary:** Military kit insurance policyholders or their nominated beneficiaries filing a claim
- **Secondary:** TMHCC claims administrators who receive the created Pega case

Happy Path (User Journey)

```

Load page → Step 1 (Your Details + Address)
           → Step 2 (What Happened + Loss Location)
           → Step 3 (Items Affected)
           → Step 4 (Supporting Evidence – file upload)
           → Step 5 (Review + Legal Declarations)
           → Submit → Pega case created → Confirmation screen
  
```

2. High-Level Architecture

Component Design Principles

1. **Single responsibility per file** — each utility module handles one concern (upload, validation, localization, etc.)
2. **Lift state to root** — all `formData` and `errors` live in `index.tsx`; sections receive only the slice they need via props
3. **Localization first** — no string is hardcoded in JSX; everything goes through the `l` object
4. **PCore abstraction** — all Pega API calls are isolated in `utils/` and accessed via `(window as any).PCore.*`
5. **Zero external state libraries** — pure React `useState` / `useEffect`; no Redux, Zustand, or Context API
6. **Validation on blur, enforce on Next** — fields show errors only after the user has touched them; the “Continue” button runs a full step validation

Architecture Diagram



Data Flow on Submit

```
User clicks Submit
→ getClaimFormValidationErrors(formData, 1)
→ if errors → setCurrentStep(1), return
→ createClaimCase(apiPath, caseTypeID, formData)
    → PCore.getRestClient().invokeCustomRestApi(POST)
    → Returns { success, ID }
→ attachUploadedFilesToCase(caseId, attachments)
    → PCore.getRestClient().invokeCustomRestApi(POST /cases/{id}/attachments)
→ setShowConfirmationView(true)
→ clearAttachmentIdsInSession()
→ ClaimConfirmationView.getPageDataAsync(D_ClaimConfirmationInfo)
```

3. File Structure Reference

TMHCC_DXLibrary_ClaimForm/	
— index.tsx	Root component – state, orchestration, rendering
— types.ts	TypeScript interfaces: ClaimFormData, ClaimFormErrors, UploadedAttachment
— config.json	Pega design-time component metadata and configurable props
— localizations.json	All default text strings and numeric constants
— mock.ts	Storybook prop defaults
— mockListValues.ts	Storybook mock for PCore.getDataPageUtils()
— demo.stories.tsx	Storybook story – sets up all PCore mocks and renders component
— demo.test.tsx	Basic smoke test
— doc-style.css	CSS stylesheet used when rendering the PDF documentation
— sections/	
— YourDetailsSection.tsx	Step 1: Personal info + policyholder address
— WhatHappenedSection.tsx	Step 2: Loss date, type, location, description
— ItemsAffectedSection.tsx	Step 3: Item type, description, prices, purchase date
— SupportingEvidenceSection.tsx	Step 4: File upload with drag-drop and document type tagging
— ReviewSection.tsx	Step 5: Collapsible summary + legal declarations
— ClaimConfirmationView.tsx	Post-submit confirmation with rich-text from Pega
— DiscardChangesModal.tsx	Modal triggered by Cancel button
— PolicyAddress.tsx	(Shared sub-component for address fields)
— InfoIcon.tsx	SVG info icon used across all section headers
— utils/	
— useLocalization.ts	React hook: reads Pega localization service into stable `l` object
— claimFormValidation.ts	Pure validation – all field validators + step/full-form runners
— claimUtils.ts	Creates the Pega claim case via REST API
— listValuesUtils.ts	Fetches LOV dropdowns (D_LOVList) and country list
— attachmentUploadUtils.ts	Uploads files to Pega attachment API
— caseAttachmentUtils.ts	Links uploaded attachment IDs to the created case
— attachmentSessionUtils.ts	sessionStorage persistence for uploaded attachment IDs
— fileValidationUtils.ts	Validates file types (PDF/JPG/PNG) before upload
— countryConfig.ts	Postal code label key map per country ISO code
— clientIpUtils.ts	Fetches client public IP (best-effort, uses ipify.org)
— claimFormStateUtils.ts	Attempts to close the browser window/tab on discard
— pegaErrorUtils.ts	Extracts human-readable messages from Pega API error shapes
— portalElementUtils.ts	Hides Pega shell UI elements during form load
— usePortalMask.ts	Hook: masks the portal for 1 s to prevent layout flash
— turnstileUtils.ts	(Legacy – no longer used by the component)
— styles/	
— styles.ts	Styled-components wrapper – applies styleResolver output
— baseStyles.ts	All BEM CSS for the form (layout, fields, buttons, etc.)
— themeStyles.ts	Theme variant overrides (currently minimal)
— personaStyles.ts	Persona variant overrides
— styleResolver.ts	Picks the correct style block based on styleVariant prop

4. Component Props and Design-Time Configuration

Props Interface

```
interface TmhccDxLibraryClaimFormProps extends PConnFieldProps {
  caseTypeID?: string; // Pega case type ID to create
  createClaimApiPath?: string; // REST API path for case creation
}
```

`PConnFieldProps` is the standard Pega DX SDK interface that provides: - `getPConnect` — function returning the connector object used to call `getLocalizationService()` - Standard display props (`displayMode` , `label` , etc.)

Design-Time (config.json) Properties

These are the properties a Pega author can configure in App Studio when placing the component on a page:

Property	Label	Format	Default	Purpose
<code>title</code>	Form title	TEXT	—	Display title (currently unused in JSX but available)
<code>caseTypeID</code>	Claim case type ID	TEXT	TMH-MCS-Work-MilitaryKitClaim	The Pega case type created on submit
<code>createClaimApiPath</code>	Create claim REST API path	TEXT	/api/application/v2/cases	Path passed to <code>invokeCustomRestApi</code> for case creation

Note: All text labels, validation messages, data page names and other strings that were previously configurable props have been moved to `localizations.json` and are managed through Pega's localization system. This keeps the App Studio designer interface clean.

Runtime-Only Props (from `PConnFieldProps`)

Prop	Type	Description
<code>getPConnect</code>	<code>() => PConnectorBase</code>	Pega DX SDK connector — provides localization service access
<code>displayMode</code>	string	Display mode hint from Pega (e.g. <code>LABELS_LEFT</code>)

Prop	Type	Description
<code>styleVariant</code>	<code>'theme' 'persona'</code>	Controls which CSS block is applied
<code>styleMode</code>	<code>'default' ...</code>	Style mode for variant selection

5. Form Steps, Fields and Validation Rules

Step 1 — Your Details

Section component: `YourDetailsSection.tsx`

Field	Type	Key in formData	Required	Validation
Policy Number	text input	<code>policyNumber</code>	Yes	Must match <code>^(?:TRI-\d{6})\$</code> (e.g. TRI-123456)
Product / Scheme	select	<code>schemeOrProduct</code>	Yes	Non-empty selection
Service Number	text input	<code>serviceNumber</code>	No	If entered, must match <code>^[A-Z]{2}\d{8}\$</code> (e.g. SA12345678)
Relationship to Policyholder	select	<code>relationship</code>	Yes	Non-empty selection
First Name	text input	<code>firstName</code>	Yes	2–100 characters
Last Name	text input	<code>lastName</code>	Yes	2–100 characters
Email Address	email input	<code>email</code>	Yes	Matches <code>^[^\\s@]+@[^\\s@]+\\.[^\\s@]+\$</code>
Phone Number	tel input	<code>phoneNumber</code>	Yes	Exactly 11 digits
Country	select	<code>policyCountry</code>	Yes	Non-empty selection
Address Line 1	text input	<code>policyAddressLine1</code>	Yes	Non-empty
Address Line 2	text input	<code>policyAddressLine2</code>	No	Free text

Field	Type	Key in formData	Required	Validation
City / Town	text input	policyCity	Yes	Non-empty
Postcode / Zip / Postal	text input	policyPostalCode	Yes	Alphanumeric (with space); label adapts per country

Country-aware postal code label logic: When `policyCountry` is `GB`, the label reads “Postcode”; for `US`, “Zip Code”; for `CA / JP`, “Postal Code”. The label key is constructed as `${country}_PCLabel` in localizations.

Address reset behavior: Changing the country clears `policyPostalCode`, `policyCity`, and `policyAddressLine1` so stale values from a previous country are never submitted.

Step 2 — What Happened

Section component: `WhatHappenedSection.tsx`

Field	Type	Key in formData	Required	Validation
Date of Loss	date input	dateOfLoss	Yes	Required; cannot be in the future
Loss Type	select	lossType	Yes	Non-empty selection
Other Loss Type	text input	otherLossType	Conditional	Required when <code>lossType === 'OTH'</code>
Reported to Police	select	theftReported	Conditional	Required when <code>lossType === 'TH'</code>
Crime Reference Number	text input	crimeReferenceNumber	Conditional	Required when <code>lossType === 'TH' AND theftReported === 'Yes'</code>
Description	textarea	description	Yes	10–1000 characters; live character counter

Field	Type	Key in formData	Required	Validation
Where did this happen	radio	<code>lossLocationType</code>	Yes	PHA (policyholder address) or DL (different location)
Loss Country	select	<code>lossCountry</code>	If DL	Non-empty
Loss Address Line 1	text input	<code>lossAddressLine1</code>	If DL	Non-empty
Loss Address Line 2	text input	<code>lossAddressLine2</code>	No	Free text
Loss City	text input	<code>lossCity</code>	If DL	Non-empty
Loss Postcode	text input	<code>lossPostalCode</code>	If DL	Country-aware validation

PHA auto-copy behavior: When “Same as policyholder address” is selected, a `useEffect` immediately copies all five policy address fields into the corresponding loss address fields. The user sees a hint banner explaining this.

Theft conditional fields: The theft section (police report + crime reference) is shown dynamically only when `lossType === 'TH'`. Changing `lossType` resets `description`, `theftReported`, `crimeReferenceNumber`, and `otherLossType`.

Step 3 — Items Affected

Section component: `ItemsAffectedSection.tsx`

Field	Type	Key in formData	Required	Validation
Item Type	select	<code>itemType</code>	Yes	Non-empty selection
Claimed Amount (£)	decimal input	<code>claimedAmount</code>	Yes	Non-empty; cannot exceed Purchase Price
Item Description	textarea	<code>itemDescription</code>	Yes	5–500 characters; live character counter
Purchase Price (£)	decimal input	<code>purchasePrice</code>	Yes	Non-empty

Field	Type	Key in formData	Required	Validation
Date Purchased	date input	<code>datePurchased</code>	Yes	Required; cannot be in the future; cannot be after Date of Loss

Cross-field validation: When `purchasePrice` changes, the `claimedAmount` error is re-evaluated immediately so users see feedback as they type the purchase price.

Step 4 — Supporting Evidence

Section component: `SupportingEvidenceSection.tsx`

Feature	Detail
Drag-and-drop area	Hidden <code><input type="file"></code> positioned over the dropzone
Accepted formats	PDF, JPG/JPEG, PNG only (validated by MIME type + extension)
Max file size	10 MB per file (enforced by Pega's upload API)
Max files	10 files per claim (configurable via <code>MaxAttachmentsCount</code> localization key)
Document type tagging	Each uploaded file requires a document type (e.g. Receipt, Police Report)
"Other" type	When "Other Evidence" is selected, a free-text specification field appears
Upload indicator	Pega <code>Progress</code> spinner while upload is in flight; dropzone disabled
Deduplication	Uploaded files are deduplicated by ID before being added to state
Session persistence	Attachment IDs saved to <code>sessionStorage</code> to survive minor page refreshes

Step 5 — Review and Declaration

Section component: `ReviewSection.tsx`

Displays a read-only summary of all four steps. Each section is an **expandable accordion card** with an Edit button that returns the user to that step.

Legal declarations (mandatory):

Declaration	Key	Requirement
Fraud Prevention Notice	<code>fraudPreventionNotice</code>	Checkbox must be checked — Submit button stays disabled until checked
Claim Accuracy Confirmation	<code>claimAccuracyConfirmation</code>	Checkbox must be checked — Submit button stays disabled until checked

The Submit button is only enabled when: `!isSubmitting && !isUploading && fraudPreventionNotice && claimAccuracyConfirmation`.

Confirmation Screen

Section component: `ClaimConfirmationView.tsx`

Feature	Detail
Rich-text content	Fetched from Pega data page <code>D_ClaimConfirmationInfo</code> using <code>getPageDataAsync</code> ; <code>field</code> <code>pyConfirmationNote</code> rendered as trusted HTML via <code>dangerouslySetInnerHTML</code>
Case reference display	The created case ID (display portion after last space) is shown to the user
Save reference button	Downloads a <code>.txt</code> file containing the claim reference
Return to Home button	Navigates to <code>ReturnHomePageLink</code> if it is a valid <code>https: URL</code> ; otherwise calls <code>onClose()</code>

6. Validation System — Deep Dive

Architecture

The entire validation system lives in `utils/claimFormValidation.ts`. It is a pure module with no React imports — all functions are deterministic given their inputs.

```
// Entry points
getClaimFieldValidationError(field, formData, l) → string // single field
getStepValidationErrors(step, formData, l) → ClaimFormErrors // all fields on a step
getClaimFormValidationErrors(formData, l) → ClaimFormErrors // all steps (used on
Submit)
```

Caching Strategy

```
let _cachedL: LocalizationMap | undefined;
let _cachedValidators: Partial<Record<keyof ClaimFormData, FieldValidator>> | undefined;

const buildValidators = (l: LocalizationMap) => {
  if (l === _cachedL && _cachedValidators) return _cachedValidators;
  // rebuild only when `l` reference changes
};
```

Because `useLocalization` uses `useMemo([getPConnect])`, the `l` reference is stable for the entire session. Validators are compiled exactly once: all regular expressions (email, phone, postal, policy number, service number) are compiled to `RegExp` objects at build time and reused.

Validator Factory Pattern

All validators are **factory functions defined at module level**. The `buildValidators` function simply assembles the map. This keeps the cognitive complexity of any single function below the SonarJS threshold of 45.

```
// Example: phone validator factory
const phoneValidator = (l, phoneRx, phoneLen) =>
  (v) => {
    if (!v) return String(l['ErrPhoneRequired']);
    return phoneRx.test(v) ? '' : String(l['ErrPhoneFormat']).replace('{1}',
      String(phoneLen));
  };

// Usage in buildValidators:
const phoneLen = l['PhoneLength']; // number (normalized by useLocalization)
const phoneRx = new RegExp(`^\\d{${phoneLen}}$`);
_cachedValidators = {
  phoneNumber: phoneValidator(l, phoneRx, phoneLen),
  ...
};
```

Validation Triggers

Trigger	Behavior
<code>onBlur</code> on a field	<code>handleFieldBlur</code> calls <code>getClaimFieldValidationError</code> for that single field

Trigger	Behavior
Change on <code>datePurchased</code>	Validated immediately on change (date inputs don't have a natural blur on mobile)
Change on <code>purchasePrice</code>	Re-validates <code>claimedAmount</code> cross-field dependency immediately
Change on <code>lossType</code>	Clears <code>description</code> , <code>theftReported</code> , <code>crimeReferenceNumber</code> , and <code>otherLossType</code> errors
Continue button	<code>getStepValidationErrors</code> for the current step; blocks navigation if any error
Submit button	<code>getClaimFormValidationErrors</code> across all steps; redirects to step 1 if errors found

7. Pega and PCore Integration Points

All Pega SDK calls go through `window.PCore`. The component never imports PCore directly — it accesses it as `(window as any).PCore` to remain compatible across Pega Infinity versions.

Integration Map

Purpose	PCore Call	File
Read localization strings	<code>getPConnect().getLocalizationService().getLocalizedText(key)</code>	<code>useLocalization</code>
Load LOV dropdowns	<code>PCore.getDataPageUtils().getDataAsync(D_LOVList, ...)</code>	<code>listValuesUtils</code>
Load country list	<code>PCore.getDataPageUtils().getDataAsync(D_CountryList, ...)</code>	<code>listValuesUtils</code>
Fetch confirmation content	<code>PCore.getDataPageUtils().getPageDataAsync(D_ClaimConfirmationInfo, ...)</code>	<code>ClaimConfirmati</code>

Purpose	PCore Call	File
Upload file	<code>PCore.getRestClient().invokeCustomRestApi('/api/application/v2/attachments/upload', POST)</code>	<code>attachmentUpload</code>
Create case	<code>PCore.getRestClient().invokeCustomRestApi(createClaimApiPath, POST)</code>	<code>claimUtils.ts</code>
Attach files to case	<code>PCore.getRestClient().invokeCustomRestApi('/api/application/v2/cases/{id}/attachments', POST)</code>	<code>caseAttachmentU</code>
Load hero image	<code>PCore.getAssetLoader().getSvcImageUrl(key)</code>	<code>index.tsx</code>

LOV Data Page Structure (D_LOVList)

The LOV data page returns a flat list of all dropdown options across all categories:

Pega Field	Mapped To	Usage
<code>pyKeyString</code>	<code>key</code>	Option value submitted to Pega
<code>pyActualValue</code>	<code>label</code>	Option display text
<code>pyCategory</code>	category key	Used to group into <code>LIST_CATEGORIES</code>
<code>pyRowNumber</code>	sort order	Options are sorted before grouping

LIST_CATEGORIES constants:

```
export const LIST_CATEGORIES = {
  PRODUCT: 'Product',
  RELATIONSHIP: 'Relationship',
  LOSS_TYPE: 'LossType',
  LOSS_LOCATION: 'LossLocation',
  GENERIC_YES_NO: 'GenericYesNo',
  ITEM_TYPE: 'ItemType',
  EVIDENCE_TYPE: 'EvidenceType'
};
```

Country Data Page Structure (D_CountryList)

Pega Field	Mapped To
ISOCODE	key (e.g. "GB")
Name	label (e.g. "United Kingdom")
DialingCode	dialingCode (e.g. "+44")

Claim Case Request Body

```
{
  caseTypeID: "TMH-MCS-Work-MilitaryKitClaim",
  content: {
    ClaimAccuracyConfirmation: boolean,
    IsFromWebForm: true,
    FraudPreventionNotice: boolean,
    PolicyNumber: string,
    Product: string,
    ServiceNumber: string,
    RelationshipWithPolicyHolder: string,
    UserDetails: {
      EmailAddress, FirstName, LastName, PhoneNumber,
      PolicyAddress: { pyCity, pyCountry, pyPostalCode, pyStreetAddress, pyStreetAddress2 }
    },
    ClaimInfo: {
      CrimeReferenceNumber, IsTheftReportedToPolice: boolean,
      LossDate, LossDescription, LossType, OtherLossType,
      WhereDidThisHappen,
      LossAddress: { pyCity, pyCountry, pyPostalCode, pyStreetAddress, pyStreetAddress2 }
    },
    ItemsAffected: [{
      ClaimAmount, Description, ItemType, PurchaseDate, PurchasePrice
    }]
  }
}
```

Loss address resolution: If `lossLocationType === 'PHA'`, the policy address fields are sent as the loss address automatically before constructing the request body.

AssetLoader Fallback Accessor

```
const assetLoader =
  (window as any).PCore?.getAssetLoader?.() ??
  (window as any).AssetLoader ??
  (window as any).PCore?.AssetLoader;
```

This three-level fallback handles different Pega Infinity versions and deployment configurations where the AssetLoader may be exposed differently.

8. Localization System

How It Works

The component's `useLocalization` hook reads every key from `localizations.json` and attempts to override each value with Pega's runtime localization:

```
export function useLocalization(getPConnect): LocalizationMap {
  return useMemo(() => {
    const defaults = localizations.fields; // from localizations.json
    const locService = getPConnect()?.getLocalizationService?.();
    if (!locService?.getLocalizedText) return defaults;

    const result = { ...defaults };
    for (const key of Object.keys(defaults)) {
      const translated = locService?.getLocalizedText(key);
      if (translated !== null) {
        const defaultVal = defaults[key];
        if (typeof defaultVal === 'number') {
          // Pega may return "11" (string) or "PhoneLength" (key-as-fallback)
          // Always coerce back to number using the JSON default as fallback
          const num = Number(translated);
          result[key] = isNaN(num) ? defaultVal : num;
        } else {
          result[key] = translated;
        }
      }
    }
    return result;
  }, [getPConnect]);
}
```

Why numeric normalization matters: Pega's `getLocalizedText` always returns a `string`. For keys like `PhoneLength: 11`, if Pega returns the key name itself as a fallback (when the localization isn't configured in the application's locale bundle), `String.slice(0, "PhoneLength")` would evaluate as `slice(0, NaN)` which produces `""` — making the phone field appear non-editable. The numeric normalization prevents this.

Configuring Strings in Pega

In App Studio or Dev Studio, navigate to the component's Localization Rule. Each key in `localizations.json` can be overridden per language/locale without a code deployment.

Data Page Names in Localizations

The following keys control which Pega data pages are fetched:

Key	Default Value	Purpose
LovDataPageName	D_LOVList	Dropdown options for all list fields
ConfirmationDataPageName	D_ClaimConfirmationInfo	Confirmation page content after submit
CountriesDataPageName	D_CountryList	Country list for address dropdowns

Moving these from `config.json` to `localizations.json` allows different Pega environments (dev/test/prod) to point to different data pages without touching component configuration.

Hero Image Key

```
"HeroImageKey": "WEBWB!CLAIMWEBFORMHOMEPAGEIMAGE!PNG"
```

This is the content management key passed to `PCore.getAssetLoader().getSvcImageUrl(key)`. It must exactly match the key of the image rule in Pega's Content Management.

9. Attachment Upload System — End to End

Upload Flow

```
User selects file(s) in dropzone
↓
validateFileTypes(files, allowedLabel)           [fileValidationUtils.ts]
  ↓ rejected files → show error; valid files continue
uploadAttachments(validFiles)                     [attachmentUploadUtils.ts]
  → Promise.all: each file POSTed to
    /api/application/v2/attachments/upload
    body: FormData { content: File, name: filename }
  ↓ returns { success, attachment: { id, name, size, docType } }
saveAttachmentIdsInSession(ids)                   [attachmentSessionUtils.ts]
  → sessionStorage key: tmhcc.claimForm.uploadAttachmentIds
↓
setFormData({ attachments: deduped })
```

Attachment-to-Case Linking (on Submit)

```
createClaimCase(...) → returns { ID: "TMH-MCS-W-1001 1001" }
↓
getCaseDisplayId(ID) → "1001" (part after last space)
↓
attachUploadedFilesToCase(fullCaseId, attachments)
→ POST /api/application/v2/cases/{fullCaseId}/attachments
→ body: {
  attachments: [
    { type: "File", category: docType, ID: attachId, name: otherDocType }
  ]
}
→ if docType === 'OE': category = "File", name = otherDocType
→ else: category = docType, name = ""
```

File Constraints

Constraint	Value	Where Enforced
Accepted types	PDF, JPG, JPEG, PNG	<code>fileValidationUtils.ts</code> — MIME + extension check
Max file size	10 MB	Pega's upload API (server-side)
Max files	10 (configurable via <code>MaxAttachmentsCount</code>)	<code>handleFileUpload</code> in <code>index.tsx</code>
Document type required	Yes	<code>handleNext</code> on Step 4 validates all attachments

Session Persistence

Attachment IDs are saved to `sessionStorage` as a JSON array under the key `tmhcc.claimForm.uploadAttachmentIds`. This means that if the user accidentally refreshes the browser, the uploaded files are not lost (they remain in Pega's attachment store referenced by their IDs). On successful submission or discard, the session key is cleared via `clearAttachmentIdsInSession()`.

10. Hero Image Loading via AssetLoader

The left-hand hero image panel background is loaded from Pega's Content Management system using the AssetLoader API.

Why AssetLoader Instead of a URL

Using `PCore.getAssetLoader().getSvcImageUrl(key)` has several advantages over a static URL: - The image is served through Pega's CDN and caching infrastructure - Image URLs can be changed in Pega without a code deployment - The same image key works across dev, test, and production environments - No hardcoded external URLs in the component code

Effect Implementation

```
useEffect(() => {
  const imageKey = String(l['HeroImageKey']).trim();
  if (!imageKey) return; // skip if key is empty (also prevents Storybook
    issues)
  let isActive = true;

  const loadHeroImage = async () => {
    const assetLoader =
      (window as any).PCore?.getAssetLoader?.() ??
      (window as any).AssetLoader ??
      (window as any).PCore?.AssetLoader;
    if (!assetLoader?.getSvcImageUrl) return;
    try {
      const imageUrl = await assetLoader.getSvcImageUrl(imageKey);
      if (isActive) setHeroImageUrl(imageUrl as string);
    } catch {
      // image unavailable - hero panel renders with fallback background color
    }
  };

  void loadHeroImage();
  return () => { isActive = false; }; // cleanup prevents stale setState
}, [1]);
```

The `isActive` flag is the standard pattern for preventing React state updates after a component has unmounted or after the effect has been cleaned up.

11. Security Controls

Input Sanitization

Field	Sanitization Method
Phone number	<code>onChange</code> strips all non-digit characters: <code>.replace(/\D/g, '')</code>
Postal code	<code>onChange</code> strips everything except <code>[a-zA-Z0-9]</code>

Field	Sanitization Method
Claimed/purchase amount	<code>onChange</code> strips non-numeric except <code>.</code> ; prevents multiple decimal points
Policy number	Validated against strict regex; no stripping needed

URL Safety Check

Both `DiscardChangesModal` and `ClaimConfirmationView` use an `isSafeUrl` guard before performing a `window.location.href` navigation:

```
const isSafeUrl = (url: string): boolean => {
  try { return new URL(url).protocol === 'https:'; } catch { return false; }
};
```

This prevents `javascript:` or `data:` protocol injection from a misconfigured localization value.

dangerouslySetInnerHTML in ClaimConfirmationView

The confirmation page renders `pyConfirmationNote` from the Pega data page as raw HTML. This is intentional — the content is a Pega correspondence rule authored by admins, not user-submitted content. A comment in the code documents this trust decision:

```
/* trusted: confirmationHtml is sourced from a Pega correspondence rule, not user input */
dangerouslySetInnerHTML={{ __html: confirmationHtml }}
```

If the source of this content ever changes, a sanitization library (e.g. DOMPurify) should be added.

Client IP Collection

`clientIpUtils.ts` calls `https://api.ipify.org?format=json` with a 5-second timeout to obtain the client's public IP. This is sent as a `RemoteIP` header on the case creation request. The call is best-effort: if it fails (blocked by CSP, ad blocker, network error), the header is simply omitted and the claim submission continues normally.

Portal Mask

On initial render, `usePortalMask` hides Pega portal shell elements (navigation, header) for 1 second to prevent a layout flash where the portal chrome appears before the form finishes mounting. This is a UX control, not a security control.

12. Accessibility Implementation

Feature	Implementation
Form labels	Every input is wrapped in a <code><label></code> element — clicking the label text focuses the input
Error messages	<code><small role="alert"></code> triggers screen-reader announcement on error appearance
Step navigation	Completed step cards have <code>role="button"</code> , <code>tabIndex={0}</code> , <code>aria-label="Go back to step N: Name"</code> , and <code>aria-current="step"</code> on the active step
Info icon button	<code>aria-label</code> toggles between “Show info” and “Hide info” based on state
Review accordion	<code>aria-expanded</code> reflects open/close state
Focus management	On step change, <code>stepContentRef.current?.focus()</code> moves focus to the new step content area (the wrapper div has <code>tabIndex={-1}</code>)
No focus ring on <code>.claim-form</code>	The programmatically-focused wrapper has <code>.claim-form:focus { outline: none; }</code> to suppress the default browser focus ring, which appeared as a jarring blue border on page load
Required fields	Marked with <code>*</code>
Progress spinner	Pega’s <code><Progress /></code> component is used during upload and submit — it is accessible by default

13. Style Architecture

Styling Approach

The component uses **styled-components** (via Pega’s `@pega/cosmos-react-core` styled-components integration) with custom BEM-style class names. All styles are scoped to `.claim-form` and `.claim-form-page` prefixes.

Style Files

File	Purpose
<code>styles.ts</code>	Re-exports the <code>StyledTmhccDxLibraryClaimFormWrapper</code> and <code>ClaimFormGlobalStyles</code> — both are styled-components
<code>styles/baseStyles.ts</code>	Master CSS block — all layout, typography, fields, buttons, review, confirmation, responsive
<code>styles/themeStyles.ts</code>	Overrides for <code>styleVariant === 'theme'</code>
<code>styles/personaStyles.ts</code>	Overrides for <code>styleVariant === 'persona'</code>
<code>styles/styleResolver.ts</code>	Picks the correct CSS block based on <code>styleVariant</code> and <code>styleMode</code> props

Layout Structure

```

.claim-form-page                (flex row)
├── .claim-form-page__image      (50% width, sticky, hero background image)
│   └── .claim-form__image-overlay (contact info panel overlaid on image)
└── .claim-form-page__panel      (50% width, flex column)
    ├── .claim-form__step-nav     (step cards row)
    ├── .claim-form              (scrollable form body, flex 1)
    │   └── <section>             (each step section)
    └── .claim-form__footer       (Cancel/Back/Continue/Submit actions)

```

Responsive Breakpoints

Breakpoint	Behavior
<code>> 1300px</code>	Full two-column layout
<code>901px-1300px</code>	Step cards become more compact (smaller text, icons hidden)
<code>≤ 900px</code>	Stacks vertically: image becomes 240px tall banner above the form; grid becomes single column
<code>≤ 480px</code>	Step cards go full-width stacked; step icons hidden

Color Palette

Token	Hex	Usage
Navy	#001b3d	Primary text, section index circles, button
Dark navy	#1A3D6B	Focus borders, headings, info box accent
Blue highlight	#7fc3ea	Active/complete step card backgrounds
Light blue	#84C2EA	Info icon color
Background	#EFF9FD	Right panel background
Error red	#ef4444	Error messages, required star
Cancel red	#da2128	Step card complete icon background

14. Developer Guide — How to Make Changes

Adding a New Form Field

1. Add the field to `ClaimFormData` interface in `types.ts`
2. Add the default value to `INITIAL_FORM_DATA` in `index.tsx`
3. Add the label and placeholder to `localizations.json`
4. Add the input/select to the appropriate section component (pass via props if needed)
5. Add a validator in `claimFormValidation.ts` — add it to `buildValidators` and to the relevant `STEP_FIELDS` entry
6. Add it to the Review display in `ReviewSection.tsx` if it should appear in the summary
7. Add it to the API request body in `claimUtils.ts` if it should be sent to Pega

Changing Validation Rules

All rules are in `utils/claimFormValidation.ts`. The regex patterns (`EmailPattern`, `PolicyNumberPattern`, `ServiceNumberPattern`, `PostalCodePattern`, `PhoneLength`) are driven by localizations — change them in `localizations.json` or via Pega's localization system without a code change.

For structural changes (new cross-field dependencies, new conditional requirements), add a factory function at module level and reference it in `buildValidators`.

Adding a New Dropdown (LOV)

1. Create a new category in Pega's `D_LOVList` data page with items tagged with the new category name
2. Add the category key to `LIST_CATEGORIES` in `listValuesUtils.ts`
3. Pass the new options to the relevant section via props in `index.tsx`

Changing the Pega Data Page Names

Do not change them in code. Configure them in Pega's localization system for the keys

`LovDataPageName` , `ConfirmationDataPageName` , and `CountriesDataPageName` .

Updating the Hero Image

Change the `HeroImageKey` value in Pega's localization system. The value must exactly match the key of the image rule in Pega Content Management (format: `WEBWB!<IMAGENAME>!<EXT>`).

Deploying to a New Pega Environment

1. Build the component store: `npm run build` inside `compstore/`
2. Import the generated ZIP into Pega using the DX Components import tool
3. Configure the component on a portal page in App Studio
4. Set `caseTypeID` and `createClaimApiPath` in the component's properties
5. Ensure the localization rules (`D_LOVList` , `D_CountryList` , `D_ClaimConfirmationInfo`) exist in that environment
6. Add the hero image to Content Management using the key in `HeroImageKey`

Key Anti-Patterns to Avoid

Anti-pattern	Why	Instead
Hardcoding strings in JSX	Breaks localization	Use <code>l['key']</code>
Calling <code>PCore.*</code> directly in a component	Untestable; breaks Storybook	Wrap in a util function
Using <code>useEffect</code> with <code>formData</code> as a dependency	Runs on every field change	Use specific field values or <code>1</code>
Storing large objects in <code>sessionStorage</code>	Session storage is limited	Only store IDs, not file blobs

15. Understanding the Pega DX Custom Component Pattern (Beginner Guide)

This section is for developers who are new to the Pega DX Component SDK and want to understand how this component fits into the Pega ecosystem.

What is a Pega DX Custom Component?

Pega Infinity (version 8.x and later) ships with a UI layer called “Cosmos” — a React-based design system. The **DX Component SDK** allows teams to build their own React components that plug into Pega’s portal and case management shell as if they were native.

Think of it like this: Pega gives you a portal framework (authentication, navigation, case creation) and you inject your custom React component into a specific page or case type.

The Three Parts of Every DX Component

```
1. config.json      ← "What settings can an author configure in App Studio?"
2. index.tsx       ← "What does the component render and do at runtime?"
3. withConfiguration(...) ← "Glue that connects Pega props to your component"
```

`config.json` is the component’s contract with Pega. It declares: - The component’s name, label, and type (`Template`) - Which subtypes it can be used on (`PAGE` , `CASE`) - Which properties are configurable in App Studio (the designer UI) - Default values for those properties

`index.tsx` is your normal React component — `useState` , `useEffect` , `JSX`, all the usual React patterns. The only Pega-specific things are: 1. The component receives a `getPConnect` prop (a function that returns the Pega connector) 2. You wrap the export with `withConfiguration(...)` so Pega can inject its props 3. You call Pega APIs via `window.PCore.*`

`withConfiguration` is a Higher-Order Component (HOC) from `@pega/cosmos-react-core` . It bridges Pega’s internal property system to your component’s props. Without it, Pega wouldn’t know how to pass the configured values (from `config.json`) to your component.

How the Component Gets Rendered

```
Pega Portal Page
├─ Pega renders a "Template" region
│   └─ Looks up component by componentKey = "TMHCC_DXLibrary_ClaimForm"
│       └─ Calls withConfiguration(TmhccDxLibraryClaimForm)
│           with props: { getPConnect, caseTypeID, createClaimApiPath, ... }
│               └─ Your React component renders
```

What is `getPConnect` ?

`getPConnect` is a factory function provided by Pega that returns the **Pega Connector** for the current component instance. The connector is your gateway to Pega's runtime services:

```
const connector = getPConnect();
connector.getLocalizationService() // → getLocalizedText(key)
connector.getDataPageUtils()      // → getDataAsync(...)
// etc.
```

Pega does NOT give you a static object — it gives you a function that creates a fresh connector. That's why `useLocalization` calls `getPConnect()` inside the `useMemo`, not once at module level.

What is `PCore`?

`window.PCore` is Pega's global runtime object — analogous to `window.jQuery` in an older web stack. It provides:

Namespace	Purpose
<code>PCore.getRestClient()</code>	Make authenticated REST API calls through Pega's proxy (handles CSRF, authentication)
<code>PCore.getDataPageUtils()</code>	Fetch data pages (read-only data from Pega's clipboard/data model)
<code>PCore.getAssetLoader()</code>	Load images and assets from Pega's Content Management system
<code>PCore.getLocaleUtils()</code>	Locale and i18n utilities

We access it as `(window as any).PCore` rather than importing it because Pega injects it into the global scope at runtime — it is not an npm package that the component bundles.

What is `localizations.json` ?

It serves two purposes: 1. **Default values** — the English strings used when no Pega localization override exists 2. **TypeScript type source** — `export type LocalizationMap = typeof localizations.fields` gives us a fully-typed `1` object with IntelliSense on every key

In production, a Pega administrator can override any key via a "Localization Rule" in Dev Studio or App Studio, enabling multi-language support without a code change.

The `withConfiguration` Lifecycle

```
Pega detects page render
→ finds component in DX Component library
→ reads config.json
→ resolves configured prop values
→ calls withConfiguration(YourComponent) with those values
→ YourComponent renders with Pega-supplied props
→ component calls PCore.* for dynamic data
```

How to Debug in Pega

- In browser DevTools console, type `window.PCore` to inspect the live Pega runtime object
- All `PCore.getRestClient().invokeCustomRestApi()` calls appear in the Network tab
- `PCore.getDataPageUtils().getDataAsync()` calls also appear in Network as XHR to Pega's data page endpoint
- To test localization: check

`getPConnect().getLocalizationService().getLocalizedText('someKey')` in the console

16. Storybook and Local Development

Running Storybook

```
cd compstore
npm install
npm run storybook
```

Open `http://localhost:6006` and navigate to **TmhccDxLibraryClaimForm > BaseTmhccDxLibraryClaimForm**.

How Storybook Mocks Work

Since Pega's runtime (`window.PCore`) isn't available in a browser dev server, `demo.stories.tsx` sets up all required mocks before rendering the component:

Mock	What it simulates
<code>PCore.getRestClient().invokeCustomRestApi</code>	File upload returns a fake attachment ID; case creation returns <code>CLM-1234</code>
<code>PCore.getDataPageUtils().getDataAsync</code>	Returns LOV and country data from <code>mockListValues.ts</code> (local JSON fixture)
<code>PCore.getLocaleUtils().getLocaleValue</code>	Pass-through

Mock	What it simulates
<code>PCore.getAssetLoader().getSvcImageUrl</code>	Returns a Cloudinary URL for the hero image
<code>getPConnect().getLocalizationService()</code>	Returns values from <code>localizations.json</code> directly (no Pega runtime needed)

Storybook hero image key: `localizations.json` has `HeroImageKey` set to a real Pega key, which would resolve to nothing in Storybook. The story overrides it via `STORYBOOK_OVERRIDES`:

```
const STORYBOOK_OVERRIDES = { HeroImageKey: 'STORYBOOK_HERO_IMAGE' };
```

The mock `getSvcImageUrl` ignores the key and always returns the Cloudinary URL, so the hero image displays correctly in Storybook.

17. Appendix — Localizations Key Reference

All keys are in `localizations.json → fields`. Numeric keys retain their type; all others are strings.

Navigation & Actions

Key	Default	Usage
<code>Cancel</code>	“Cancel”	Cancel button
<code>Continue</code>	“Continue”	Next step button
<code>Back</code>	“Back”	Previous step button
<code>Submit</code>	“Submit”	Final submission button
<code>Processing...</code>	“Processing...”	Submit button label while in flight

Discard Modal

Key	Default
<code>DiscardModalHeading</code>	“Cancel claim form?”
<code>DiscardModalBody</code>	“Are you sure you want to cancel...”
<code>DiscardModalConfirm</code>	“Yes, Cancel Form”
<code>DiscardModalDismiss</code>	“No, Continue”

Help Panel (Hero Image Overlay)

Key	Default
HelpTextHeader	"We're here to help!"
HelpDescription	Subtitle text
HelpTelephoneValue	"0808 175 4908"
HelpEmailValue	"triclaims@tmhcc.com"
HelpOverseasValue	"+ 44 808 175 4908"
HelpSupportHoursValue	"Monday – Friday, 9:00am – 5:00pm"

Step Labels

Key	Default
Step 1	"Step 1"
Your Details	"Your Details"
Step 2	"Step 2"
What Happened	"What Happened"
Step 3	"Step 3"
Items Affected	"Items Affected"
Step 4	"Step 4"
Evidence	"Evidence"

Form Field Labels

Key	Default
Policy Number	"Policy Number"
Product	"Product"
Service Number	"Service Number"
Relationship to Policyholder	"Relationship to Policyholder"
First Name	"First Name"
Last Name	"Last Name"

Key	Default
Email Address	"Email Address"
Phone Number	"Phone Number"
Date of Loss	"Date of Loss"
Loss Type	"Loss Type"
Description	"Description"
Item Type	"Item Type"
Claimed Amount	"Claimed Amount"
Item Description	"Item Description"
Purchase Price	"Purchase Price"
Date Purchased	"Date Purchased"
Country	"Country"
Address Line 1	"Address Line 1"
Address Line 2	"Address Line 2"
CityTown	"City / Town"

Placeholders

Key	Default
PolicyNumberPH	"TRI-123456"
ServiceNumberPH	"SA12345678"
FNPH	"James"
LNPH	"Thompson"
EmailPH	"j.thompson@mod.uk"
PNPH	"07890481548"
AmountPH	"£850.00"
SelectOptionPH	"Select option"
CountryPH	"Select country"
DatePlaceholder	"dd/mm/yyyy"

Validation Patterns (Configurable Regexes)

Key	Default
EmailPattern	<code>^[^\s@]+@[^\s@]+\.[^\s@]+\$</code>
PostalCodePattern	<code>^[a-zA-Z0-9]+\$</code>
PolicyNumberPattern	<code>^(?:TRI-\d{6})\$</code>
ServiceNumberPattern	<code>^[A-Z]{2}\d{8}\$</code>

Numeric Constants

Key	Default	Type	Usage
PhoneLength	11	number	<code>maxLength</code> attribute and phone regex <code>^\d{11}\$</code>
NameMinLength	2	number	First/Last name minimum
NameMaxLength	100	number	First/Last name maximum and <code>maxLength</code>
DescriptionMinLength	10	number	Description minimum characters
DescriptionMaxLength	1000	number	Description <code>maxLength</code>
ItemDescriptionMinLength	5	number	Item description minimum
ItemDescriptionMaxLength	500	number	Item description <code>maxLength</code>
MaxAttachmentsCount	10	number	Upload file limit
CharCounterWarnThreshold	950	number	Character counter turns red at this count

Country-Specific Postal Code Keys

Key	Default
GB_PCLabel	"Postcode"

Key	Default
US_PCLabel	“Zip Code”
CA_PCLabel	“Postal Code”
JP_PCLabel	“Postal Code”
GB_PCErrMsg	“Postcode is required”
US_PCErrMsg	“Zip Code is required”
CA_PCErrMsg	“Postal Code is required”
PCErrMsg	“Postal Code is required”

Error Messages

Key	Default
ErrFirstNameRequired	“First name is required”
ErrLastNameRequired	“Last name is required”
ErrFNTooShort	“First Name should have atleast {1} characters”
ErrFNTooLong	“First Name must be {1} characters or fewer”
ErrPolicyNumberRequired	“Policy number is required”
PolicyNumberFormatError	“Policy number must be in the format TRI-456789...”
ErrRelationshipRequired	“Relationship to policyholder is required”
ErrEmailRequired	“Email is required”
ErrEmailInvalid	“Enter a valid email address”
ErrPhoneRequired	“Phone number is required”
ErrPhoneFormat	“Phone number must be {1} digits”
ErrServiceNumberFormat	“Service number must be in the format SA12345678...”
ErrProductRequired	“Product is required”
ErrDateOfLossRequired	“Date of loss is required”
ErrDateOfLossFuture	“Date of loss cannot be in the future”
ErrLossTypeRequired	“Loss type is required”
ErrOtherLossTypeRequired	“Please specify the loss type”

Key	Default
ErrTheftReportedRequired	"Please select whether the theft was reported..."
ErrCrimeReferenceRequired	"Crime reference number is required"
ErrDescriptionRequired	"Description is required"
ErrDescriptionTooShort	"Description must be at least {1} characters"
ErrDescriptionTooLong	"Description must be {1} characters or fewer"
ErrCountryRequired	"Country is required"
ErrAddressLine1Required	"Address line 1 is required"
ErrCityRequired	"City / Town is required"
ErrPostalCodeRequired	"Postal code is required"
ErrPostalCodeInvalid	"{1} is invalid"
ErrItemTypeRequired	"Type of item is required"
ErrItemDescriptionRequired	"Item description is required"
ErrPurchasePriceRequired	"Purchase price is required"
ErrClaimedAmountRequired	"Claimed amount is required"
ErrClaimedAmountExceedsPurchase	"Claimed amount cannot be greater than the purchase price"
ErrDatePurchasedRequired	"Date purchased is required"
ErrDatePurchasedFuture	"Purchase date cannot be in the future"
ErrDatePurchasedAfterLoss	"Purchase date must be before the Date of Loss ({1})"
ErrEvidenceTypeRequired	"Evidence type is required"
ErrEvidenceTypeSpecify	"Please specify the evidence type"
ErrConsentRequired	"Please confirm before submitting the claim"
ErrFieldRequired	"Required"

Confirmation Screen

Key	Default
ConfirmationRefPending	"Pending"

Key	Default
ConfirmationSaveRef	"Save Claim Reference"
ConfirmationReturnHome	"Return to Home Page"
ReturnHomePageLink	"https://talktotrinity.com/"

Data Page and Asset Keys (Configurable via Pega Localization)

Key	Default	Purpose
HeroImageKey	WEBWB! CLAIMWEBFORMHOMEPAGEIMAGE! PNG	Content Management key for hero background
LovDataPageName	D_LOVList	Pega data page for all dropdowns
ConfirmationDataPageName	D_ClaimConfirmationInfo	Pega data page for confirmation content
CountriesDataPageName	D_CountryList	Pega data page for country dropdown

End of documentation. Generated 2026-06-09.